

บทที่ 6

โอแอลอี (OLE – Object Linking and Embedding)

6.1 โอแอลอี (OLE – Object Linking and Embedding)

โอแอลอี เป็นลักษณะการเขียนโปรแกรมอย่างหนึ่ง โดยใช้รูปแบบของคอมมอปเจ็กต์ (COM Object) โดยจะแบ่งการทำงานออกเป็น 2 ส่วนสำคัญคือโอแอลอีเซิร์ฟเวอร์ (OLE Server) และส่วนที่เป็นโอแอลอีไคลเอนต์ (OLE Client) โดยส่วนเซิร์ฟเวอร์ทำหน้าที่ให้บริการแก่ส่วนไคลเอนต์ ซึ่งส่วนไคลเอนต์นี้จะทำการฝังตัวลงบนโปรแกรม แล้วเรียกใช้การทำงานของส่วนเซิร์ฟเวอร์ เราจะเรียกโปรแกรมที่สามารถนำวัตถุที่เป็นโอแอลอีไคลเอนต์ไปฝังนั้นว่า โอแอลอีคอนเทนเนอร์ (OLE Container)

การทำงานของโอแอลอีนั้นจะมีลักษณะดังนี้

1. การเชื่อมโยงและการฝัง (Linking and Embedding) การเชื่อมโยงและการฝังเป็นวิธีการเชื่อมต่อหรือฝังตัวของวัตถุที่เป็นโอแอลอีไคลเอนต์ที่สร้างโดยโปรแกรมโอแอลอีเซิร์ฟเวอร์ลงในเอกสาร

2. อินเพชเอกติเวชัน (In - Place Activation (Visual Editing)) เมื่อเราทำการเรียกดูวัตถุที่เป็นโอแอลอีไคลเอนต์ที่ฝัง (Embedding) ในโอแอลอีคอนเทนเนอร์ขึ้นมา การทำงานส่วนติดต่อกับผู้ใช้ของโอแอลอีคอนเทนเนอร์จะเปลี่ยนเป็นรูปแบบของโอแอลอีเซิร์ฟเวอร์เพื่อเรียกฟังก์ชันของโอแอลอีเซิร์ฟเวอร์ขึ้นมาทำงาน แต่ถ้าโอแอลอีไคลเอนต์นั้นเป็นวัตถุที่แสดงเพียงแค่การเชื่อมโยง (Linking) ส่วนติดต่อกับผู้ใช้ของโอแอลอีคอนเทนเนอร์จะไม่เปลี่ยนเป็นลักษณะดังกล่าว แต่จะเป็นการเรียกการทำงานโปรแกรมที่เป็นผู้สร้างวัตถุนั้นขึ้นมาทำงานแทน

3. ออโตเมชัน (Automation) เป็นการทำงานในลักษณะที่โปรแกรมหนึ่งสามารถเรียกฟังก์ชันการทำงานของอีกโปรแกรมหนึ่งได้ โปรแกรมที่เป็นผู้เรียกฟังก์ชันจะเรียกว่าโอแอลอีออโตเมชันไคลเอนต์ (OLE Automation Client) หรือ โอแอลอีออโตเมชันคอนโทรลเลอร์ (OLE Automation Controller) ส่วนโปรแกรมที่เป็นผู้ถูกเรียกนั้นเราจะเรียกว่า โอแอลอีออโตเมชันเซิร์ฟเวอร์ (OLE Automation Server) หรือ โอแอลอีออโตเมชันคอมโพเนนต์ (OLE Automation Component)

4. คอมพาวด์ไฟล์ (Compound Files) คอมพาวด์ไฟล์เป็นลักษณะที่ไฟล์หนึ่งไฟล์ประกอบด้วยวัตถุต่าง ๆ ในรูปแบบที่เป็นมาตรฐานประกอบกันเป็นไฟล์นั้น เช่น ภายในเอกสารของไมโครซอฟท์เวิร์ดจะประกอบด้วยวัตถุตัวอักษร, วัตถุรูปภาพ, วัตถุกราฟ, วัตถุลายมือชื่อ เป็นต้น

5. รูปแบบการส่งข้อมูล (Unifrom Data Transfer) การส่งข้อมูลระหว่างโปรแกรมที่สนับสนุนการทำงานแบบโอแอลอีนั้นจะมีรูปแบบที่เป็นมาตรฐาน ได้แก่ ใช้หลักการทำงานของคลิปบอร์ดหรือไดนามิกดาต้าเอ็กซ์เชนจ์ ซึ่งเป็นหลักการสำคัญในการนำข้อมูลมาสร้างเป็นโปรแกรมลายมือชื่อดิจิตอลของคณะผู้จัดทำ

6. ลากและวาง (Drag and Drop) คือการทำงานในรูปแบบลากแล้ววางวัตถุจากโปรแกรมหนึ่งไป อีกโปรแกรมหนึ่ง เป็นรูปแบบการส่งข้อมูลระหว่างโปรแกรมที่ใช้มากในปัจจุบัน

จากหลักการดังกล่าว คณะผู้จัดทำจะนำหลักการของโอแอลอีออโตเมชันและโอแอลอีเซิร์ฟเวอร์ มาใช้ในโครงงานดังนี้

6.1.1 โอแอลอีออโตเมชัน (OLE Automation)

โอแอลอีออโตเมชันเป็นวิธีการในการจัดการกับวัตถุของโปรแกรม (Application) ที่สนใจ โดยสามารถควบคุมได้จากนอกโปรแกรมนั้น ๆ ซึ่งในการที่จะสามารถใช้งานได้นั้น โปรแกรมที่เขียนขึ้นมา จำเป็นที่จะต้องติดต่อกับโปรแกรมที่สนใจโดยการใช้คอมอินเทอร์เฟซ (COM interface) ซึ่งการใช้ อินเทอร์เฟซก็จะประกอบไปด้วยพรอพเพอร์ตี้ (Property) และเมธอด (Method) ต่าง ๆ เราสามารถเรียก โปรแกรมที่เขียนขึ้นมาเพื่อใช้จัดการกับโปรแกรมอื่นว่าออโตเมชันไคลเอนต์ (Automation Client) เพราะ เราได้เรียกใช้เมธอดจากโปรแกรมอื่น และในแนวคิดเดียวกันเราสามารถเรียกโปรแกรมที่เราไปติดต่อด้วย ว่า ออโตเมชันเซิร์ฟเวอร์ (Automation Server) เพราะสามารถให้บริการต่าง ๆ กับเรา

อินเทอร์เฟซต่าง ๆ จะถูกประกาศไว้ในรูปแบบที่เรียกว่า ODL (Object Description Language) ซึ่งไคลเอนต์สามารถเรียกใช้ได้จากไฟล์ชนิดของไลบรารี (Type Library) โปรแกรมต่าง ๆ ที่สามารถเรียก ใช้ชนิดของไลบรารีได้จะทำการสร้างซอร์สโค้ดขึ้นมาเพื่อใช้กับออโตเมชันไคลเอนต์

การสร้างโปรแกรมให้ทำงานกับโอแอลอีออโตเมชันโดยใช้เอ็มเอฟซี (MFC) และชนิดของ ไลบรารี (Type Library) มีขั้นตอนดังนี้

1. ทำการนำชนิดของไลบรารีเข้า (Import Type Library) จากคลาสสิคัล (Class Wizard) โดยการเลือกหัวข้อออโตเมชันภายในคลาสสิคัล
2. ทำการเพิ่มคลาส (Add Class) จากชนิดของไลบรารีจากนั้นเลือกไฟล์ไลบรารีของโปรแกรมที่ต้องการจะติดต่อด้วย ในที่นี้จะเลือกไฟล์ชนิดของไลบรารีของโปรแกรมไมโครซอฟท์เวิร์ด (Microsoft Word) โดยต้องเลือกไฟล์ C:\Program Files\Microsoft Office\Office\MsWord8.olb
3. จากนั้นโปรแกรมจะทำการสร้างซอร์สโค้ด (Source Code) ของคลาสที่เราเลือกให้สร้าง ผลที่ได้ออกมาจะเป็นไฟล์ .cpp และ .h ของออโตเมชันเซิร์ฟเวอร์
4. ทำการอินคลูด (include) ไฟล์เฮดเดอร์ของออโตเมชันเซิร์ฟเวอร์
5. เริ่มการติดต่อกับออโตเมชันเซิร์ฟเวอร์ โดยการตั้งคำสั่ง AfxOleInit() เพื่อเป็นการจัดตั้งค่าเริ่มต้นต่าง ๆ
6. หลังจากนั้นจะต้องใส่คำสั่ง AfxEnableControlContainer(); เพื่อที่จะทำให้โปรแกรมสามารถ เข้าควบคุมออโตเมชันได้
7. สร้างวัตถุของโปรแกรมที่เป็นออโตเมชันเซิร์ฟเวอร์ โดยการประกาศ _Application app;

8. ทำการกำหนดโปรแกรมที่เราต้องการให้เป็นอโตเมชันเซิร์ฟเวอร์ให้กับวัตถุของอโตเมชันเซิร์ฟเวอร์ โดยใช้คำสั่ง GetActiveObject();
9. สามารถเริ่มการติดต่อกับโปรแกรมอโตเมชันเซิร์ฟเวอร์ได้

6.1.2 โอแอลอีเซิร์ฟเวอร์ (OLE Server)

โปรแกรมแบบโอแอลอีเซิร์ฟเวอร์เป็นโปรแกรมที่ทำงานเกี่ยวกับวัตถุต่าง ๆ ซึ่งวัตถุเหล่านี้จำเป็นต้องมีคลาสที่ใช้ในการสร้างวัตถุโดยโปรแกรมทั้งหมดที่ทำงานบนไมโครซอฟท์วินโดวส์จะรู้จักคลาสเหล่านี้ โดยการใช้ที่รีจิสทรีในส่วนของคีย์ HKEY_CLASSES_ROOT\CLSID ซึ่งเปรียบเสมือนกับตัวเลขที่ใช้บ่งบอกถึงคลาสทั้งหมดที่วินโดวส์รู้จักโอแอลอีเซิร์ฟเวอร์จะต้องเป็นโปรแกรมที่ให้บริการในเรื่องของการเพิ่มวัตถุลงในเอกสารของโปรแกรมที่สนับสนุนการทำงานแบบโอแอลอีและต้องสามารถให้บริการในเรื่องของการเชื่อมโยงและฝังตัววัตถุได้ด้วย

ขั้นตอนการสร้างโปรแกรมแบบโอแอลอีเซิร์ฟเวอร์มีดังนี้

1. กำหนดให้โปรแกรมทำงานแบบ SDI (Single Document Interface)
2. คลาสที่ใช้สำหรับแสดงผล หน้าจอของโปรแกรมโอแอลอีเซิร์ฟเวอร์จะสืบทอดมาจากคลาส CView เพื่อใช้ในการแสดงรูปวัตถุลายมือชื่อ (Signature Object)
3. ในเมธอด InitInstance() จะต้องเรียกใช้คำสั่ง AfxOleInit() เพื่อเริ่มต้นการทำงานของโอแอลอี
4. ทำการเพิ่มคลาสไอดี (Class ID) เพื่อเป็นการบอกระบบให้รู้ว่ามีคลาสใหม่ที่เป็นของโอแอลอีเซิร์ฟเวอร์
5. จะต้องทำการแก้ไขคลาสวิว (Class view) ที่ใช้ในการแสดงผล ในกรณีที่มีการแทรกวัตถุของลายมือชื่อลงในเอกสารใด ๆ เพื่อใช้แสดงผลของวัตถุนั้น

คลาสต่าง ๆ ที่เกิดจากการสร้างโปรแกรมแบบโอแอลอีเซิร์ฟเวอร์จะสืบทอดมาจากคลาสต่าง ๆ ดังนี้

1. CFrameWnd เป็นคลาสที่จะใช้เก็บหน้าต่างหลักของโปรแกรมในขณะที่โปรแกรมทำงานแยกต่างหากจากโอแอลอีไคลเอ็นต์
2. CWinApp เป็นคลาสที่ใช้สำหรับโปรแกรมหลัก เพื่อเป็นการกำหนดการทำงานของโปรแกรมหลัก
3. COleServerDoc เป็นคลาสที่ใช้สำหรับเก็บข้อมูลของโปรแกรมหลักเพื่อนำไปบันทึกเป็นไฟล์ได้
4. COleServerItem เป็นคลาส ที่ใช้เก็บวัตถุของไอเท็ม (Item) ต่าง ๆ ที่ถูกใช้โดยโอแอลอีไคลเอ็นต์

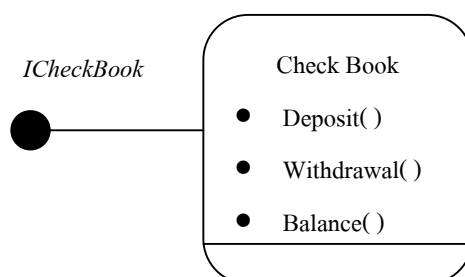
5.CView เป็นคลาสที่ใช้เก็บส่วนของการแสดงผลของหน้าจอหลักของโปรแกรม

6.COleIPFrameWnd เป็นคลาสที่ใช้จัดการกับวิซวลอีดิติง (Visual Editing) ที่เกิดจากโอแอลอีไคลเอ็นต์

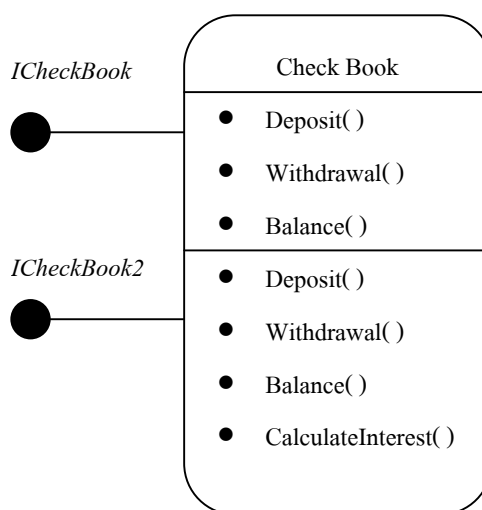
6.2 อินเทอร์เฟซ (Interface)

รูปแบบการเขียนโปรแกรมแบบคอมมอปเจ็กต์ (COM – Object) นั้นจะประกอบไปด้วยอินเทอร์เฟซ ซึ่งทำหน้าที่ติดต่อกันระหว่างสองวัตถุ (Object) ใด ๆ โดยเป็นกระบวนการที่นำเมธอด (Method) ที่เราต้องการใช้จากวัตถุ อินเทอร์เฟซต่าง ๆ จะสืบทอด (Derive) มาจากอินเทอร์เฟซ IUnknown ซึ่งจากหลักการของโอแอลอีนั้นเมื่อไคลเอ็นต์ทำการติดต่อกับอินเทอร์เฟซได้ก็จะสามารถเรียกใช้เมธอดของทางเซิร์ฟเวอร์ได้โดยผ่านทางอินเทอร์เฟซนั้น ๆ เมธอดการทำงานต่าง ๆ ของอินเทอร์เฟซนั้นจะถูกเก็บไว้ในตารางวิเทเบิล (V Table) ในรูปแบบของพอยน์เตอร์ชี้ไปที่เมธอดของเซิร์ฟเวอร์

เมื่อตัววัตถุมีการเปลี่ยนแปลงโดยการสร้างหรือเพิ่มเมธอดเข้าไปใหม่ต้องมีการสร้างอินเทอร์เฟซใหม่ซึ่งสืบทอดมาจากอินเทอร์เฟซเดิมเพื่อให้อินเทอร์เฟซใหม่สามารถเห็นเมธอดที่ทำการเพิ่มเข้าไปใหม่ได้ แต่ถ้ามีการปรับปรุงเมธอดที่มีอยู่แล้วต้องทำการเตรียมอินเทอร์เฟซใหม่ที่บรรจุเมธอดที่ปรับปรุงแล้ว แต่เราไม่สามารถสืบทอดมาจากอินเทอร์เฟซเดิมได้ เพราะชื่อของเมธอดซ้ำกัน การทำงานไม่เหมือนกัน



รูปที่ 6.1 ตัวอย่างอินเทอร์เฟซของคอมมอปเจ็กต์



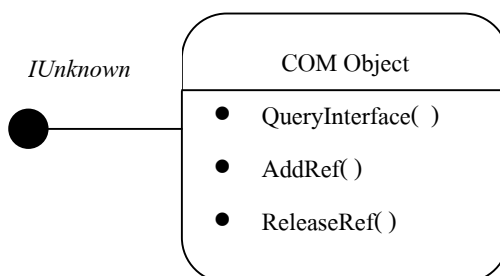
รูปที่ 6.2 ตัวอย่างแสดงการสร้างอินเทอร์เฟซใหม่เมื่อมีเมธอดใหม่

6.2.1 อินเทอร์เฟซ IUnknown

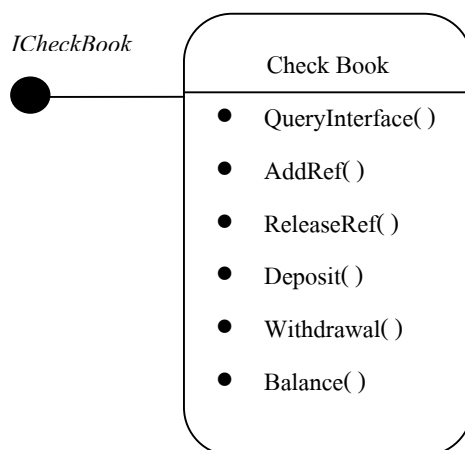
ประกอบด้วยเมธอดหลัก 3 เมธอดด้วยกันคือ

1. QueryInterface() ใช้เพื่อดูว่าวัตถุมีอินเทอร์เฟซที่เราต้องการหรือไม่
2. AddRef() ทำการเพิ่มค่าเรเฟอเรนซ์เคาน์ติง
3. ReleaseRef() ทำหน้าที่ลดค่าเรเฟอเรนซ์เคาน์ติง

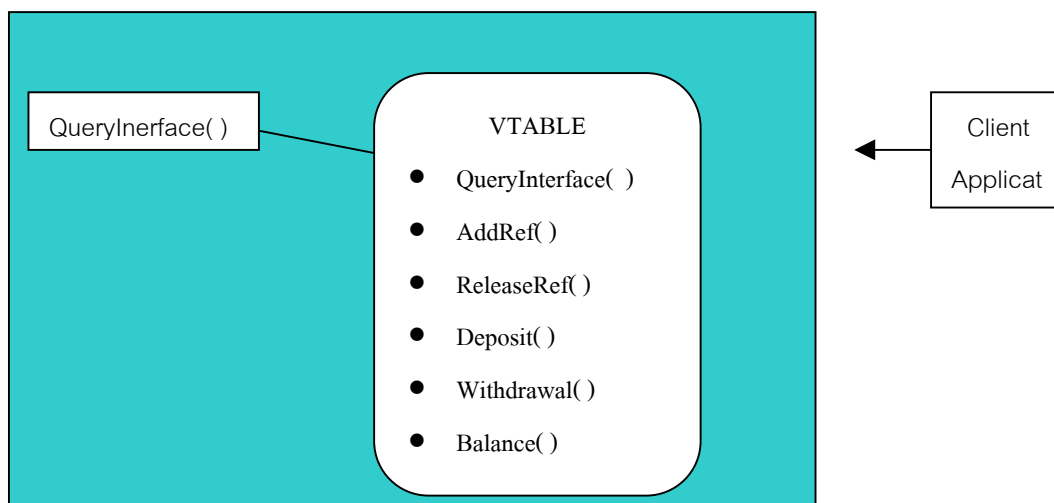
โดยเมื่อเซิร์ฟเวอร์ให้ค่าพอยน์เตอร์ที่ชี้ไปยังอินเทอร์เฟซออกมา ต้องมีการเพิ่มเรเฟอเรนซ์เคาน์ติงโดยเรียกผ่านเมธอด AddRef() เมื่อไคลเอนต์ทำงานกับเซิร์ฟเวอร์เสร็จสิ้นก็จะลดค่าเรเฟอเรนซ์เคาน์ติงลงโดยเรียกผ่านเมธอด ReleaseRef() โดยค่าของเรเฟอเรนซ์เคาน์ติงมีไว้เพื่อระบุว่าไคลเอนต์จำนวนเท่าใดที่ใช้อินเทอร์เฟซนี้บ้าง เมื่อค่าเรเฟอเรนซ์เคาน์ติงเป็นศูนย์ก็ไม่สามารถใช้เซิร์ฟเวอร์นั้นได้อีก



รูปที่ 6.3 แสดงอินเทอร์เฟซ IUnknown



รูปที่ 6.4 แสดงอินเทอร์เฟซที่สืบทอดมาจาก IUnknown



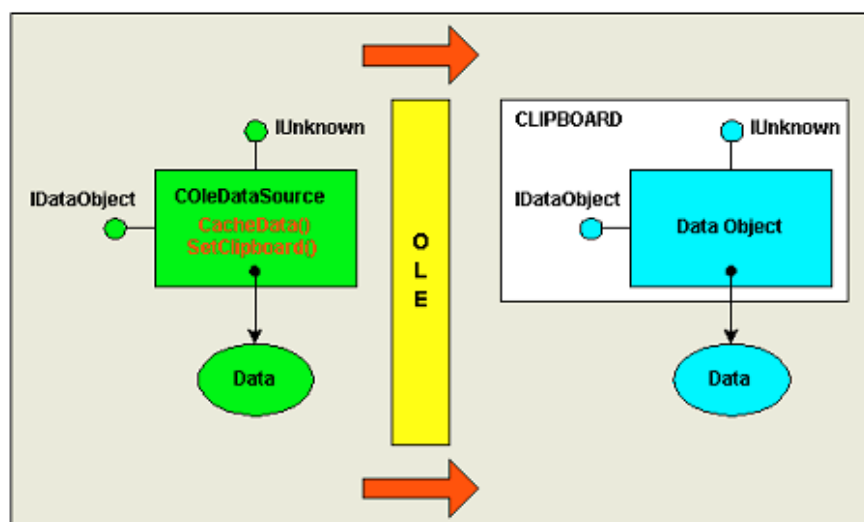
รูปที่ 6.5 แสดงลักษณะ VTABLE

6.2.2 อินเทอร์เฟซ IDataObject

วัตถุข้อมูล (Data Object) หมายถึง วัตถุต่าง ๆ ที่สนับสนุนอินเทอร์เฟซ IDataObject อินเทอร์เฟซ IDataObject เป็นอินเทอร์เฟซที่มีเมธอดในการส่งข้อมูลและบอกถึงการเปลี่ยนแปลงข้อมูลของวัตถุ โดยในการส่งข้อมูลจะต้องมีการระบุรูปแบบของข้อมูล และตัวกลางที่ใช้ในการส่งข้อมูลด้วย

6.3 คลาส COleDataObject

คลาส COleDataObject เป็นคลาสที่ใช้ในการส่งผ่านข้อมูลในหลายรูปแบบระหว่างโปรแกรม ซึ่งข้อมูลอาจมาจากคลิปบอร์ด (Clipboard) ผ่านทางการลากและวาง (Drag and Drop) โดยในความเป็นจริงข้อมูลนั้นจะได้มาจากการสร้างคลาส COleDataSource หรือ ผ่านทางอินเทอร์เฟซ IDataObject ก็ได้



รูปที่ 6.6 แสดงการทำงานของคลาส COleDataObject

ภายในคลาส COleDataObject จะมีฟังก์ชันที่ใช้ในการรับส่งข้อมูลระหว่างโปรแกรมที่สนับสนุนการทำงานในแบบโอแอลอีอยู่ โดยเราสามารถที่จะระบุประเภทของข้อมูลที่เราต้องการผ่านฟังก์ชันในคลาส COleDataObject นี้ได้ประเภทของข้อมูลดังกล่าวขณะผู้จัดทำได้อธิบายไว้ในบทคลิปปอร์ด และการส่งผ่านข้อมูลใด ๆ ก็ตามโดยใช้คลาส COleDataObject นี้เราจำเป็นต้องระบุประเภทของตัวกลางที่ใช้ในการส่งข้อมูลด้วย ตัวกลางดังกล่าวอาจเป็นไฟล์โดยใช้คลาส CFile, การใช้โกลบอลเมมโมรี (HGLOBAL) หรือใช้โครงสร้าง STGMEDIUM เป็นต้น ซึ่งฟังก์ชันที่ขณะผู้จัดทำนำมาใช้มีดังต่อไปนี้

1. void COleDataObject::Attach(LPDATAOBJECT lpDataObject, BOOL bAutoRelease = NULL) เป็นฟังก์ชันที่กำหนดจุดเริ่มต้นให้กับตัวแปรของคลาส COleDataObject ว่ารับข้อมูลในโปรแกรมที่สนับสนุนการทำงานในแบบโอแอลอีตัวใด โดยผ่านทางอินเทอร์เฟซ IDataObject ซึ่ง lpDataObject ซึ่อยู่

2. void COleDataObject::BeginEnumFormats() เป็นฟังก์ชันเพื่อเตรียมการใช้งานฟังก์ชัน GetNextFormat เพื่อเก็บค่าตำแหน่งของรูปแบบแรก

3. BOOL COleDataObject::GetNextFormat(LPFORMATETC lpFormatEtc) เป็นฟังก์ชันที่เอาตัวแปรพอยน์เตอร์ชี้ไปยังข้อมูลรูปแบบต่อไป ในกรณีที่รูปแบบของข้อมูลนั้นเป็นรูปแบบสุดท้ายจะคืนค่าออกมาเป็นเท็จ นอกนั้นจะค่าออกมาเป็นจริง

4. BOOL COleDataObject::IsDataAvailable(CLIPBOARD cfFormat, LPFORMATETC lpFormatEtc = NULL) เป็นฟังก์ชันที่ใช้เพื่อตรวจสอบว่ามีข้อมูลประเภทคลิปปอร์ดที่ระบุตาม cfFormat ภายในคลาส COleDataObject หรือไม่ ในกรณีที่รูปแบบข้อมูลดังกล่าว ฟังก์ชันคืนค่าออกมาเป็นจริง นอกนั้นจะค่าออกมาเป็นเท็จ

5. HGLOBAL COleDataObject::GetGlobalData(CLIPBOARD cfFormat, LPFORMATETC lpFormatEtc = NULL) เป็นฟังก์ชันกำหนดโกลบอลเมมโมรีเพื่อรับค่าข้อมูลประเภทคลิปปอร์ดที่ระบุตาม cfFormat ในกรณีที่กำหนดค่าโกลบอลเมมโมรีสำเร็จจะคืนค่าจริง นอกนั้นคืนค่าเท็จ

6.4 ฟอแมตทีซี (FORMATETC)

เป็นโครงสร้างข้อมูลที่ระบุรูปแบบของข้อมูลในการส่งข้อมูลของโปรแกรมที่สนับสนุนการทำงานแบบโอแอลอี โครงสร้างของฟอแมตทีซีโดยย่อมีดังนี้

```
typedef struct tagFORMATETC
```

```
{
```

CLIPFORMAT	cfFormat;	// รูปแบบของข้อมูลที่ต้องการ เช่น CF_TEXT
DVTARGETDEVICE	*ptd;	// รายละเอียดโครงสร้างของอุปกรณ์เป้าหมาย
DWORD	dwAspect;	// ตัวระบุรายละเอียดของวัตถุในคลิปปอร์ด
LONG	lindex;	// ตัวระบุขอบเขตของข้อมูลที่ต้องการ

```
        DWORD          tymped;          // ประเภทของตัวกลางในการส่งข้อมูล
    } FORMATETC, *LPFORMATETC;
```